REPUBLIC OF YEMEN
ALSAEED UNIVERSITY- TAIZ

Penetration Testing1

PART 8

Buffer Overflow Attacks

Assistant Teacher: Mubarak Altamimi.

What Is Buffer Overflow

Definition:

- Buffer overflow is a software coding error or vulnerability that can be exploited by hackers to gain unauthorized access to corporate systems.

- It is one of the best-known software security vulnerabilities yet remains fairly common.

Buffer Overflow

- A buffer is an area of **adjacent memory** locations allocated to a program or application to handle its runtime data
- Buffer overflow or overrun is a **common vulnerability** in an applications or programs that accepts more data than the allocated buffer
- This vulnerability allows the application to exceed the buffer while writing data to the buffer and **overwrite neighboring memory** locations
- Attackers exploit buffer overflow vulnerability to **inject malicious code** into the buffer to damage files, modify program data, access critical information, escalate privileges, gain shell access, etc.

Why Are Programs and Applications Vulnerable to Buffer Overflows?

- Lack of boundary checking
- Using older versions of programming languages
- Using unsafe and vulnerable functions
- Lack of good programming practices
- Failing to set proper filtering and validation principles
- Executing code present in the stack segment
- Improper memory allocation
- Insufficient input sanitization

Buffer Overflow Attack

- A buffer overflow attack takes place when an attacker manipulates the coding error to carry out malicious actions and compromise the affected system.
- The attacker alters the application's execution path and overwrites elements of its memory, which amends the program's execution path to damage existing files or expose data.

Buffer Overflow Attack

A buffer overflow vulnerability will typically occur when code:

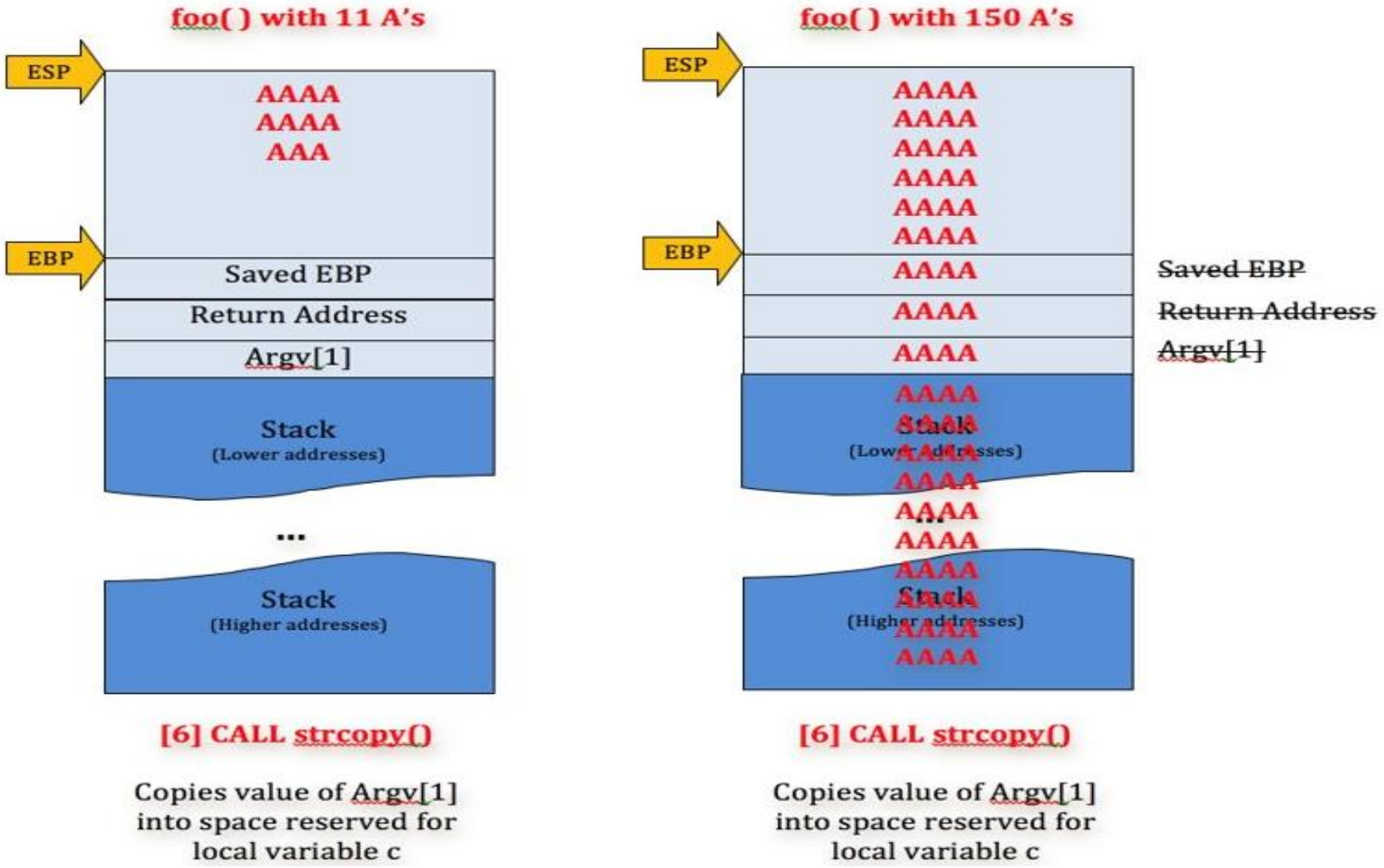
- Is reliant on external data to control its behavior
- Is dependent on data properties that are enforced beyond its immediate scope
- Is so complex that programmers are not able to predict its behavior accurately

Buffer overflow consequences

Common consequences of a buffer overflow attack include the following:

- **System crashes:** A buffer overflow attack will typically lead to the system crashing. It may also result in a lack of availability and programs being put into an infinite loop.
- **Access control loss**: A buffer overflow attack will often involve the use of arbitrary code, which is often outside the scope of programs' security policies.
- **Further security issues:** When a buffer overflow attack results in arbitrary code execution, the attacker may use it to exploit other vulnerabilities and subvert other security services.

Buffer overflow consequences



Types Of Buffer Overflow Attacks

There are several types of buffer overflow attacks that attackers use to exploit organizations' systems. The most common are:

Stack-based buffer overflows: This is the most common form of buffer overflow attack. The stack-based approach occurs when an attacker sends data containing malicious code to an application, which stores the data in a stack buffer and overwrites the data on the stack, including its return pointer, which hands control of transfers to the attacker.

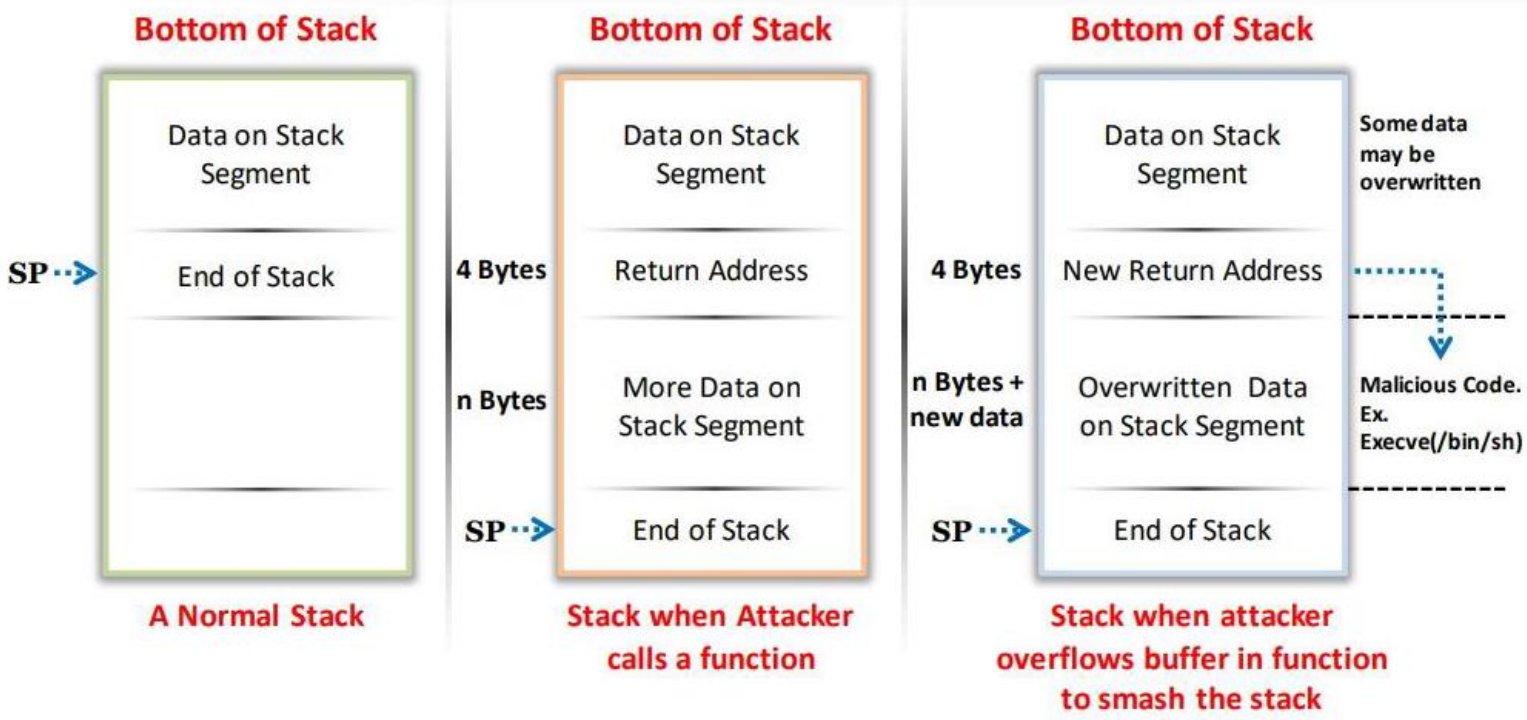
Types Of Buffer Overflow Attacks

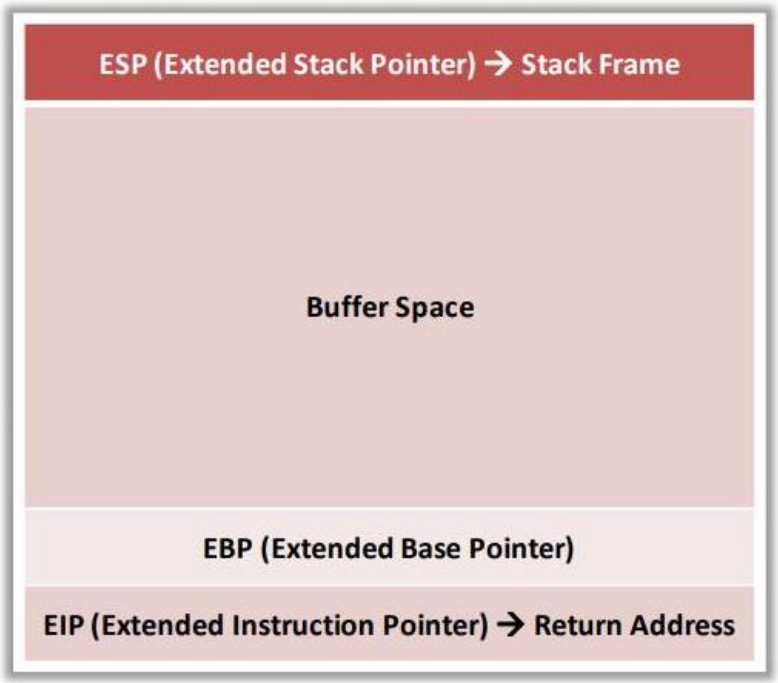
Heap-based buffer overflows: A heap-based attack is more difficult to carry out than the stack-based approach. It involves the attack flooding a program's memory space beyond the memory it uses for current runtime operations.

Format string attack: A format string exploit takes place when an application processes input data as a command or does not validate input data effectively. This enables the attacker to execute code, read data in the stack, or cause segmentation faults in the application. This could trigger new actions that threaten the security and stability of the system.

Types of Buffer Overflow: Stack-Based Buffer Overflow

- A stack is used for **static memory allocation** and stores the variables in "Last-in First-out" (LIFO) order
- There are two stack operations: **PUSH** stores the data onto the stack and **POP** removes data from the stack
- If an application is vulnerable to stack-based buffer overflow, then attackers take control of the EIP register to **replace the return address** of the function with the malicious code that allows them to gain shell access to the target system

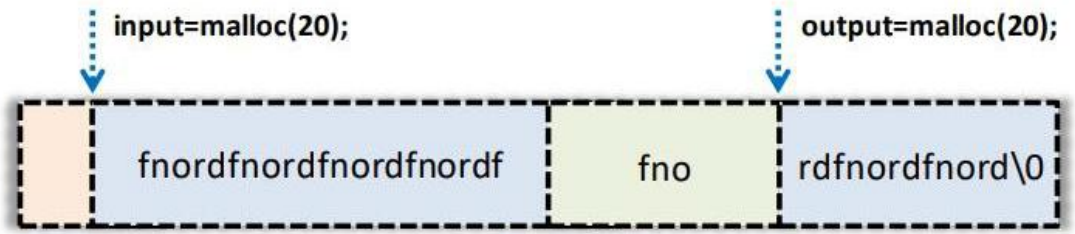




Types of Buffer Overflow: Heap-Based Buffer Overflow

- Heap memory is **dynamically allocated** at runtime during the execution of the program and it stores program data
- Heap-based overflow occurs when a block of memory is allocated to a heap, and data is written without any bounds checking
- This vulnerability leads to **overwriting dynamic object pointers**, heap headers, heap -based data, virtual function table, etc.
- Attackers exploit heap-based buffer overflow to take control of the program's execution. Unlike stack overflows, heap overflows are inconsistent and have different exploitation techniques

Heap: Before Overflow



Heap: After Overflow

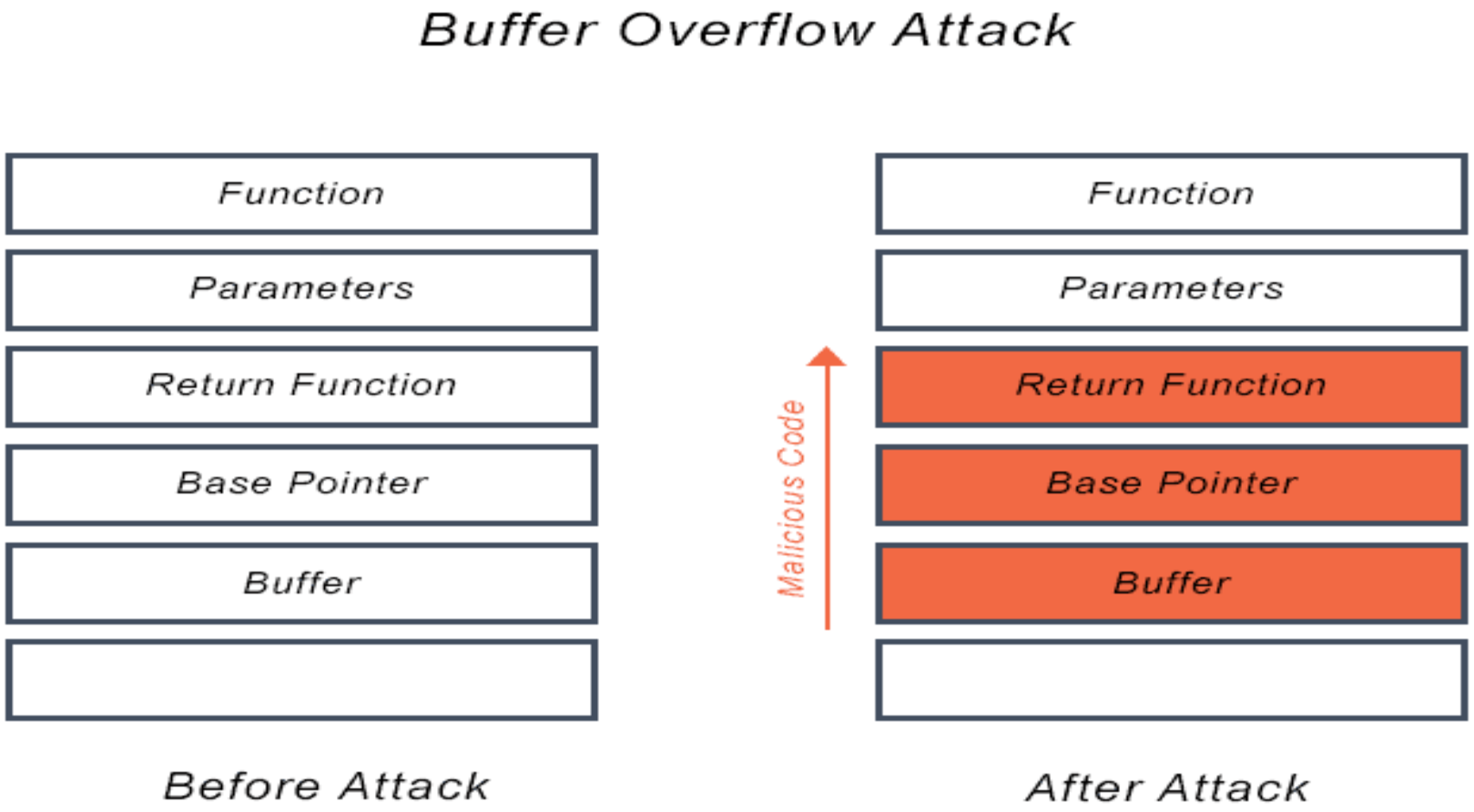
Buffer overflow exploits

- Attackers use a buffer overflow to corrupt a web application's execution stack, execute arbitrary code, and take over a machine.
- Flaws in buffer overflows can exist in both application servers and web servers, especially web applications that use libraries like graphics libraries.
- Buffer overflows can also exist in custom web application code.
- This is more likely because security teams give them less scrutiny but are less likely to be discovered by hackers and more difficult to exploit.

Buffer overflow exploits

- The buffer overflow exploit techniques a hacker uses depends on the architecture and operating system being used by their target. However, the extra data they issue to a program will contain malicious code that enables the attacker to trigger additional actions and send new instructions to the application.

Buffer overflow exploits



Windows Buffer Overflow Exploitation

Steps involved in exploiting Windows based buffer overflow vulnerability:

1. Perform spiking
2. Perform fuzzing
3. Identify the offset
4. Overwrite the EIP register
5. Identify bad characters
6. Generate shellcode
7. Identify the right module
8. Gain root access

Buffer Overflow Attack Examples

- For example, a simple buffer overflow can be caused when code that relies on external data receives a ‘gets()’ function to read data in a stack buffer. The system cannot limit the data that is read by the function, which makes code safety reliant on users entering fewer than ‘BUFSIZE’ characters. This code could look like this:

“…

```
char buf[BUFSIZE];
gets(buf);
```

Buffer Overflow Attack Examples

Other buffer overflow attacks rely on user input to control behavior then add indirection through the memory function ‘memcpy()’. This accepts the destination buffer, source buffer, and amount of bytes to copy, fills the input buffer with the ‘read()’ command, and specifies how many bytes for ‘memcpy()’ to copy.

- “...

```
char buf[64], in[MAX_SIZE];
printf("Enter buffer contents:\n");
read(0, in, MAX_SIZE-1);
printf("Bytes to copy:\n");
scanf("%d", &bytes);
memcpy(buf, in, bytes);
```

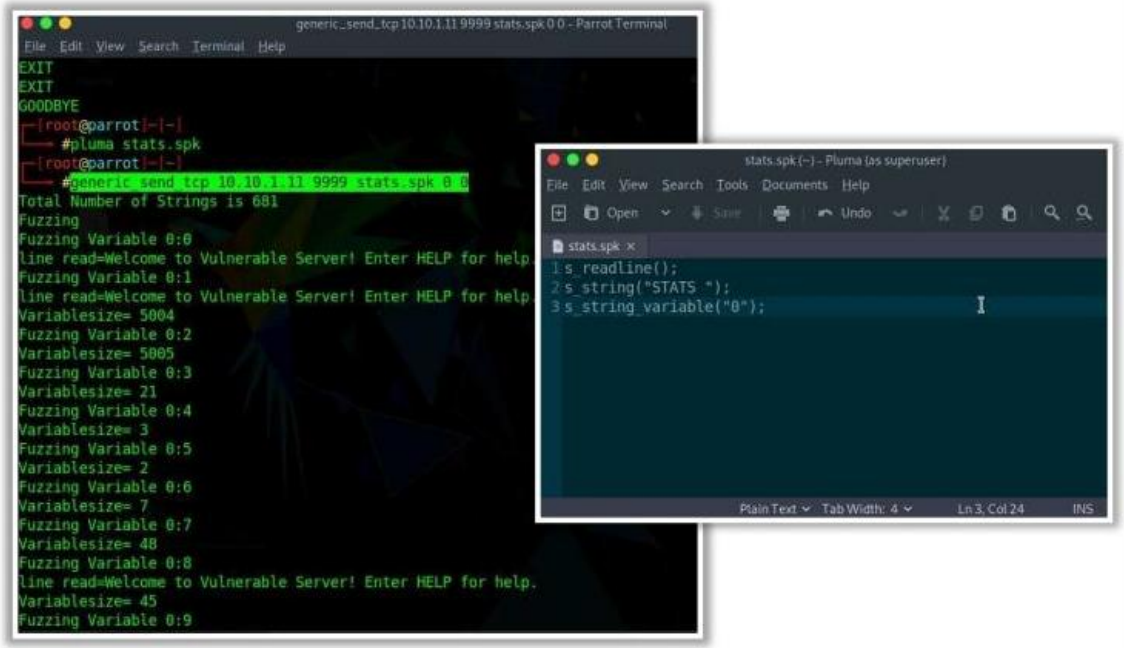
Windows Buffer Overflow Exploitation (Cont'd)

Perform Spiking

- Spiking allows attackers to send crafted TCP or UDP packets to the vulnerable server in order to make it crash
- Spiking helps attackers to identify buffer overflow vulnerabilities in the target applications

- **Step 1: Establish a connection with the vulnerable server using Netcat**

- **Step 2: Generate spike templates and perform spiking**



Windows Buffer Overflow Exploitation (Cont'd)

Perform Fuzzing

- Attackers use fuzzing to send a **large amount of data** to the target server so that it experiences buffer overflow and overwrites the EIP register
- Fuzzing helps in identifying the number of bytes required to crash the target server
- This information helps in determining the exact **location of the EIP** register, which further helps in injecting malicious shellcode

fuzz.py (~/Scripts) - Pluma

File Edit View Search Tools Documents Help

Open Save Undo

fuzz.py ×

```
#!/usr/bin/python2
import sys, socket
from time import sleep

buff = "A" * 100

while True:
    try:
            soc = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            soc.connect(('10.10.1.11', 9999))

            soc.send(('TRUN /.:/' + buff))
            soc.close()
            sleep(1)
            buff = buff + "A" * 100
    except:
            print "Fuzzing crashed vulnerable server at %s bytes" % str(len(buff))
            sys.exit()
```

```
┌─[✗]─[root@parrot]─[~]
└──╼ #cd /home/attacker/Desktop/Scripts
┌─[root@parrot]─[/home/attacker/Desktop/Scripts]
└──╼ #chmod +x fuzz.py
┌─[root@parrot]─[/home/attacker/Desktop/Scripts]
└──╼ #./fuzz.py
^CFuzzing crashed vulnerable server at 11800 bytes
┌─[root@parrot]─[/home/attacker/Desktop/Scripts]
└──╼ #
```

Windows Buffer Overflow Exploitation (Cont'd)

Overwrite the EIP Register

- Overwriting the EIP register allows attackers to identify whether the EIP register can be controlled and can be overwritten with **malicious shellcode**

overwrite.py (~/Scripts) - Pluma

File Edit View Search Tools Documents Help

Open Save Undo

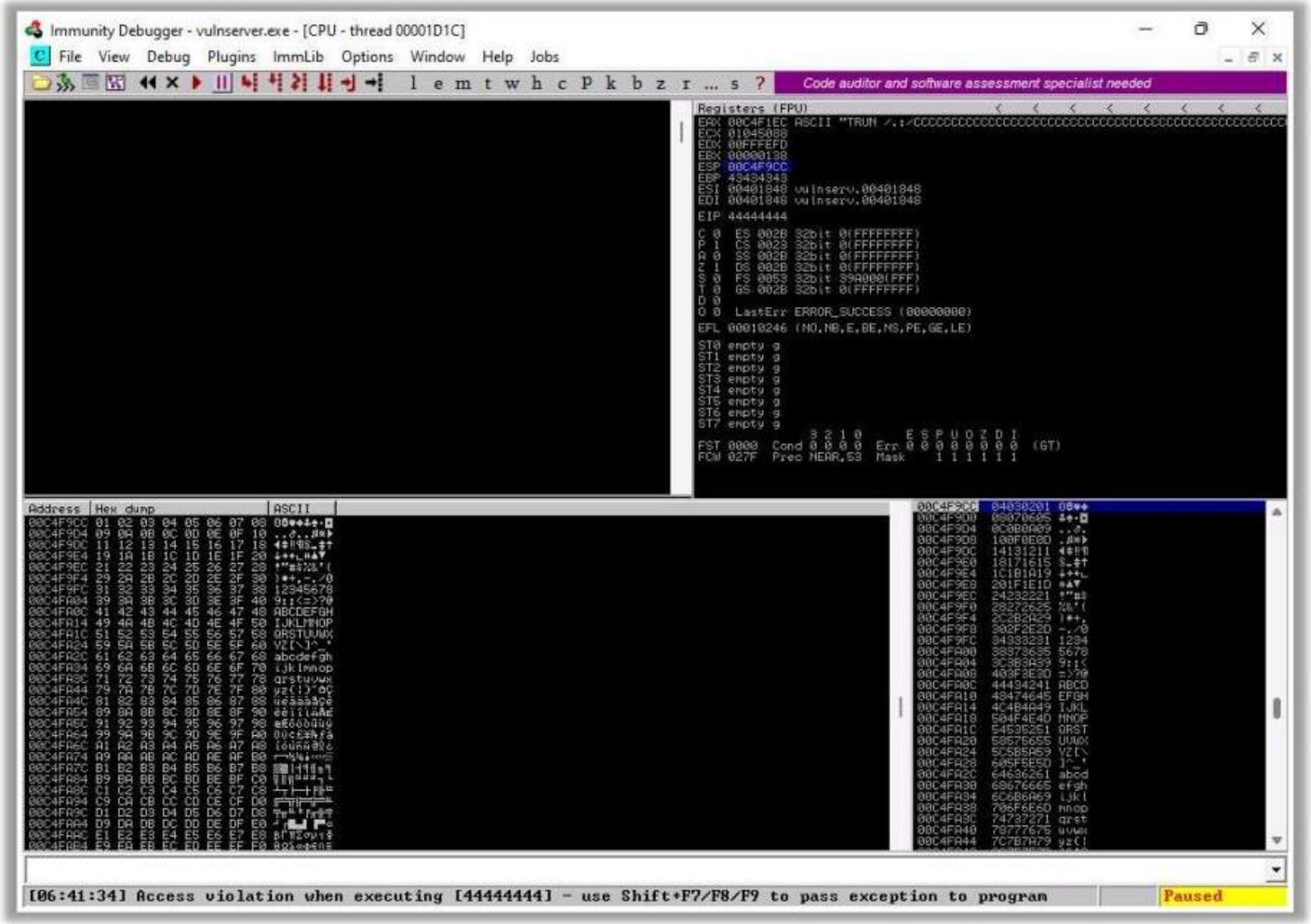
overwrite.py

```
#!/usr/bin/python2
import sys, socket

shellcode = "C" * 2003 + "D" * 4

try:
    soc = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    soc.connect(('10.10.1.11', 9999))
    soc.send(('TRUN /.:/' + shellcode))
    soc.close()
except:
    print "Error: Unable to establish connection with Server"
    sys.exit()
```

Windows Buffer Overflow Exploitation (Cont'd)

Identify Bad Characters

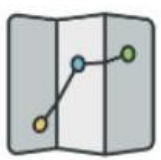
- Before injecting the shellcode into the **EIP register**, attackers identify bad characters that may cause issues in the shellcode
- You can obtain the badchars through a Google search. Characters such as no byte, i.e., "**\x00**", are badchars

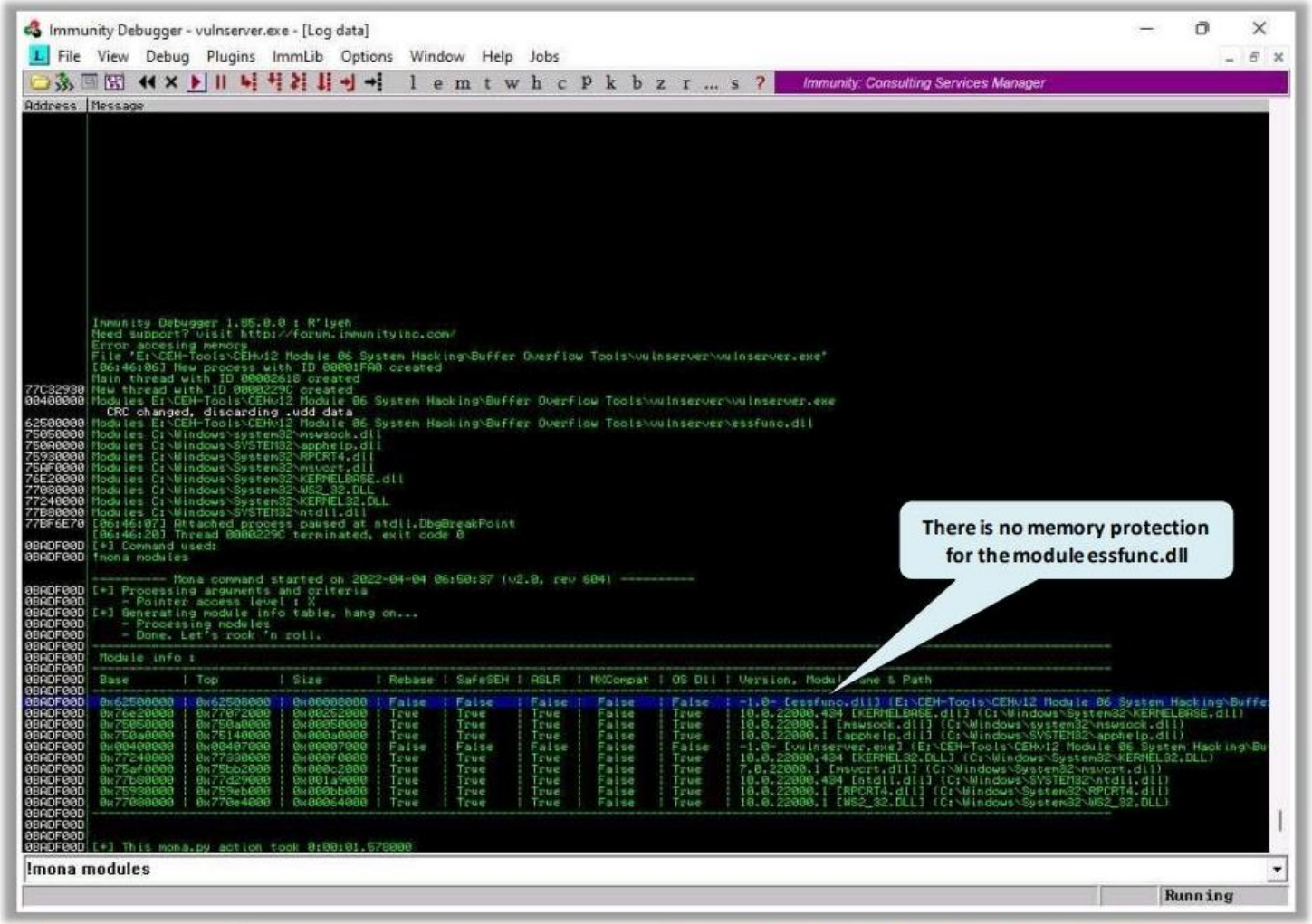


Windows Buffer Overflow Exploitation (Cont'd)

Identify the Right Module

- In this step, attackers identify the right module of the vulnerable server that **lacks memory protection**
- In Immunity Debugger, you can use scripts such as **mona.py** to identify modules that lack memory protection





Return-Oriented Programming (ROP) Attack

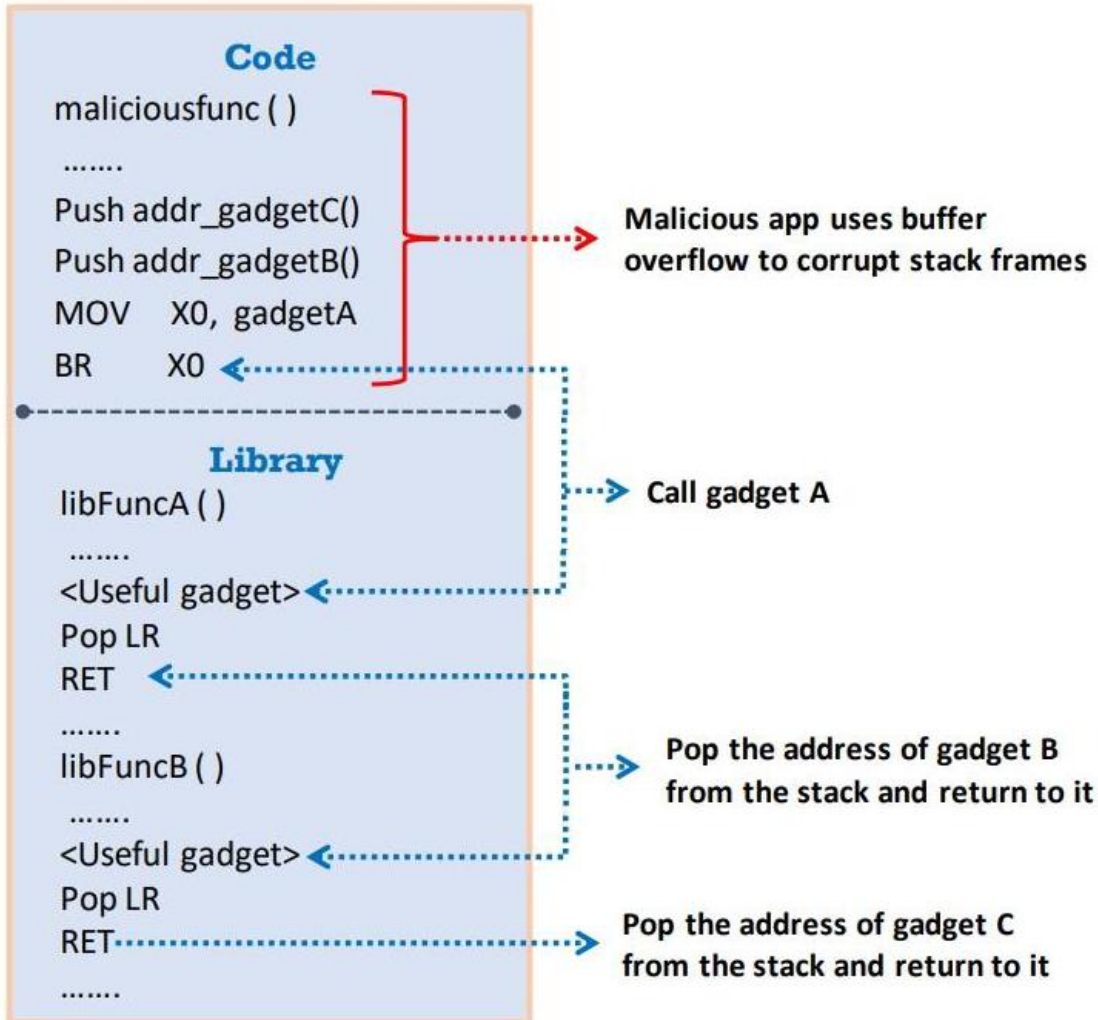
Return-oriented programming (ROP) is an **exploitation technique** used by attackers to execute arbitrary malicious code

An attacker hijacks the target **program control flow** by gaining access to the call stack and then executes arbitrary machine instructions by reusing available libraries known as gadgets

Gadgets are a collection of instructions that end with the **x86 RET instruction**

The attacker selects a **chain of existing gadgets** to create a new program and executes it with malicious intentions

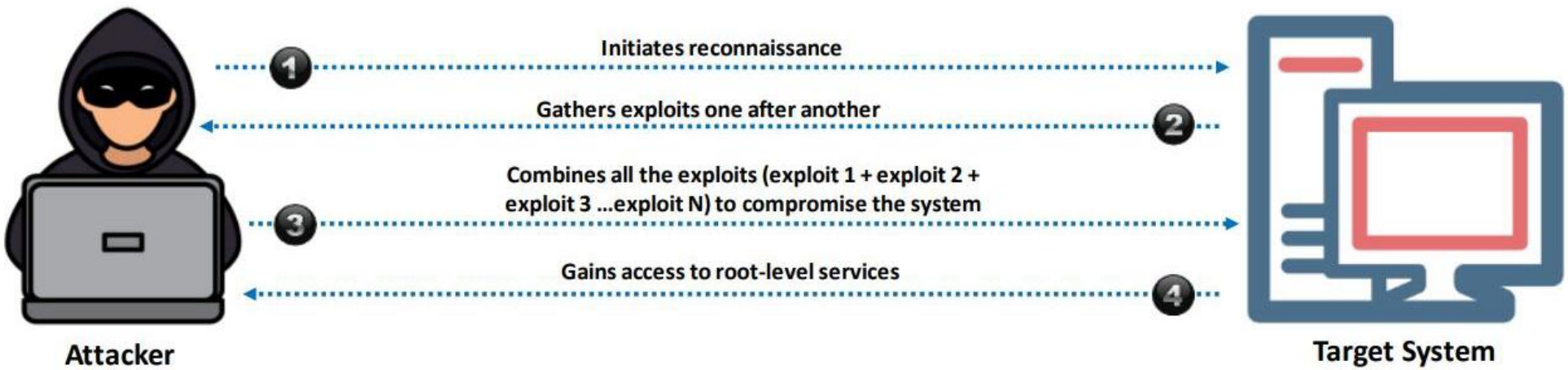
ROP attacks are very effective as they utilize available and **legal code libraries**, which are not identified by security protections such as **code signing** and executable space protection



Exploit Chaining

- Exploit chaining, also referred to as **vulnerability chaining**, is a cyberattack that combines various exploits or vulnerabilities to infiltrate and compromise the target from its root level

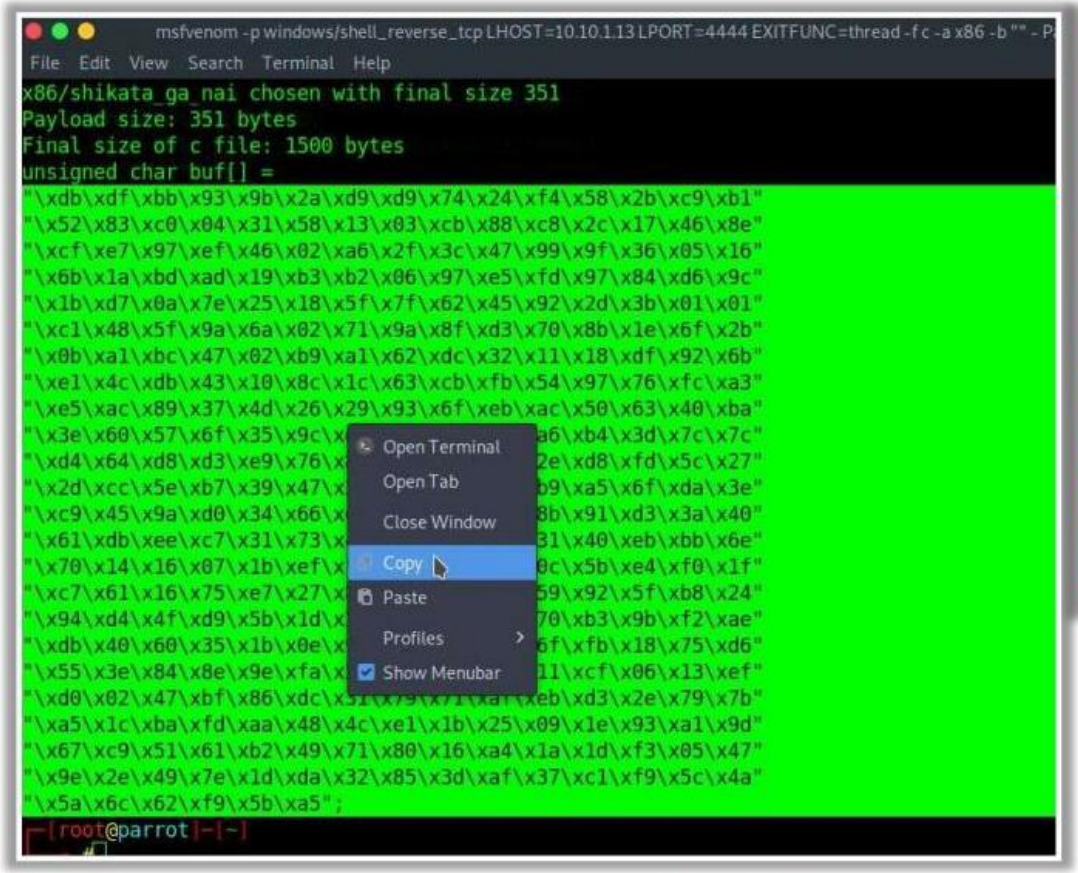
- During exploit chaining, an attacker first initiates the **reconnaissance** operation and then starts **enumerating** various **digital footprints** and underlying vulnerabilities one after another within the software or hardware of the target system

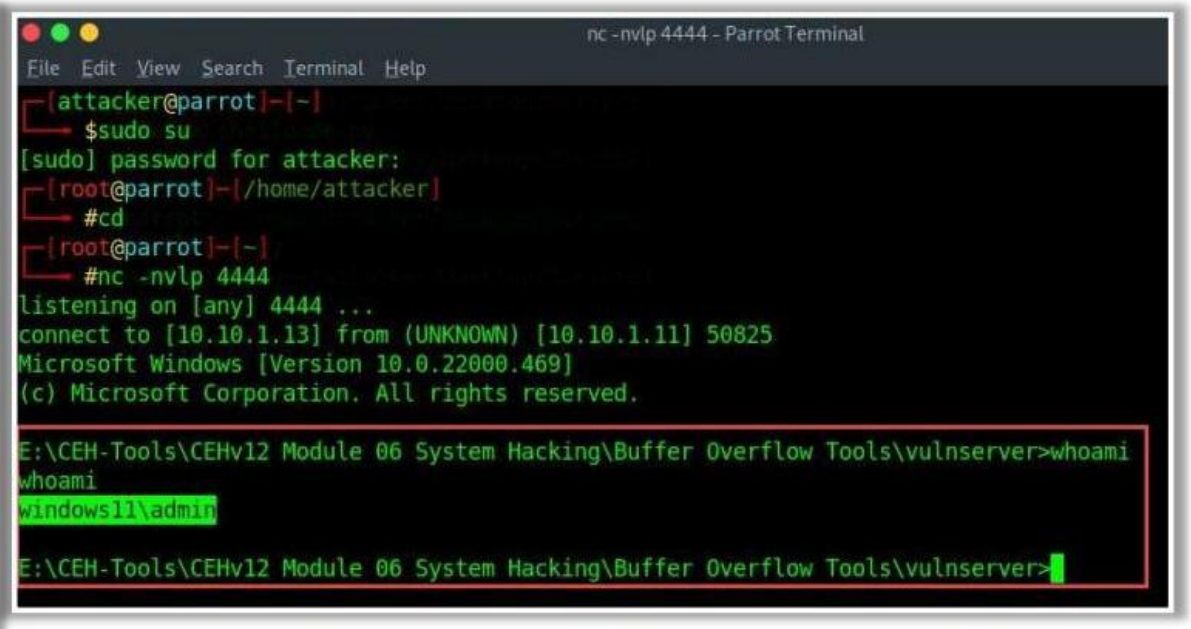


Windows Buffer Overflow Exploitation (Cont'd)

Generate Shellcode and Gain Shell Access

- Attackers use the **msfvenom command** to generate the shellcode and inject it into the EIP register to gain shell access to the target vulnerable server

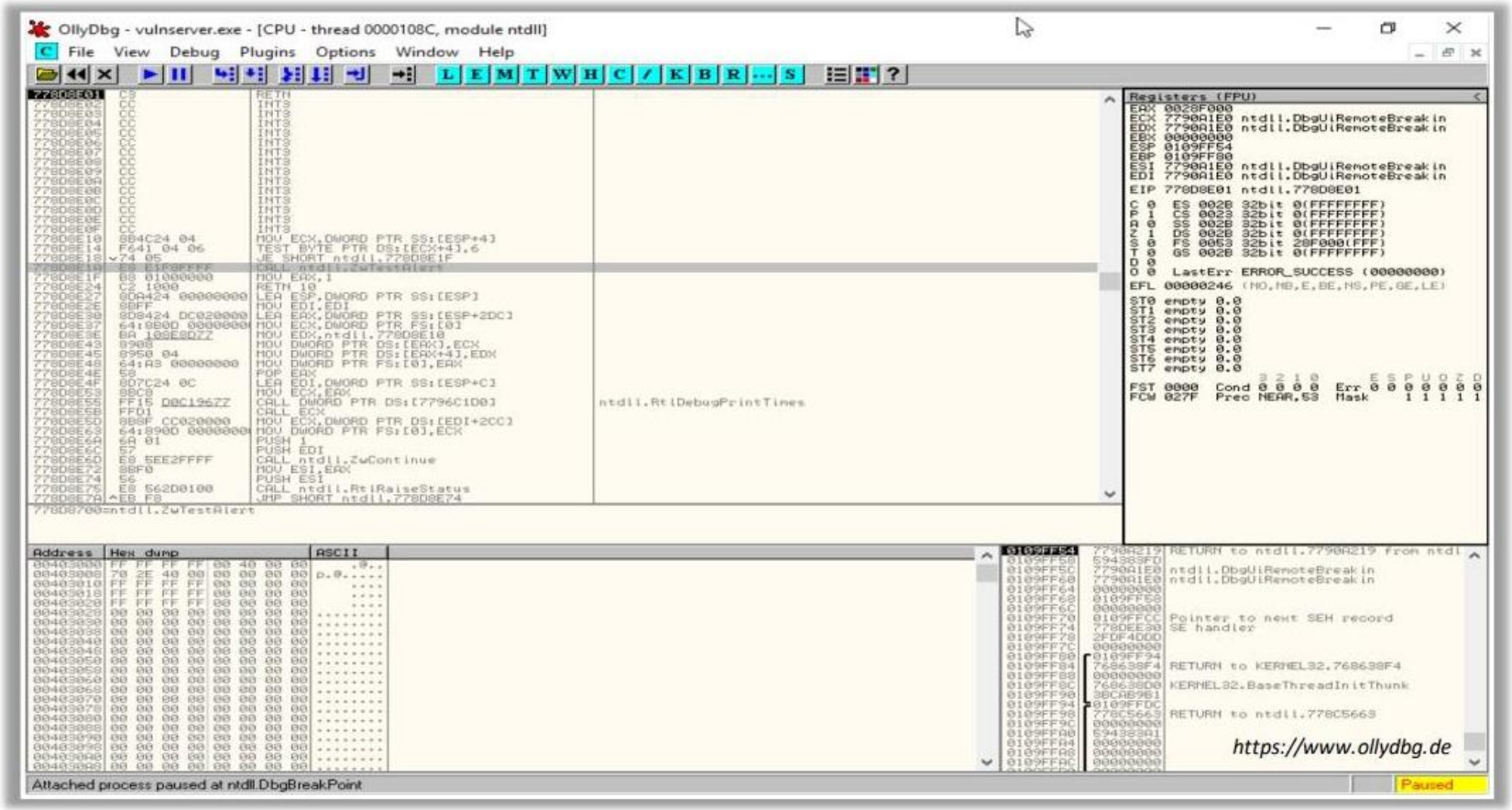




Buffer Overflow Detection Tools

OllyDbg

OllyDbg dynamically **traces stack frames** and program execution, and it logs arguments of known functions



Veracode
https://www.veracode.com

Flawfinder
https://dwheeler.com

Kiuwan
https://www.kiuwan.com

Splint
https://github.com

BOVSTT
https://github.com

Defending against Buffer Overflows

1. Develop programs by following **secure coding practices** and guidelines
2. Use the **address space layout randomization** (ASLR) technique
3. Validate arguments and **minimize code** that requires root privileges
4. Perform **code review** at the source-code level by using static and dynamic code analyzers
5. Allow the compiler to **add bounds** to all the buffers
6. Implement **automatic bounds checking**
7. Always protect the **return pointer** on the stack
8. Never allow the execution of code outside the code space
9. **Regularly patch** applications and OSes
10. Perform **code inspection** manually with a checklist to ensure that the code meets certain criteria
11. Employ **data execution prevention** (DEP) to mark the memory regions as non-executable
12. Implement **code pointer integrity** checking to detect whether a code pointer has been corrupted before it is dereferenced